

■ Modules, Errors & Exceptions

Python — Modules, Packages, Namespaces, Scope, Errors & Exception Handling

#	Topic
1	Modules
2	Importing Modules
3	Packages
4	Libraries
5	Namespaces & Scope (LEGB Rule)
6	global Keyword
7	nonlocal Keyword
8	Errors & the Exception Hierarchy
9	try / except / else / finally
10	Catching Multiple Exceptions
11	raise & assert
12	Custom Exceptions
13	Common Built-in Exceptions

1 · Modules

A **module** is simply a single `.py` file containing Python code — functions, classes, or variables — that can be imported and reused in other files.

Term	Meaning
Module	A single <code>.py</code> file
<code>import module_name</code>	Brings a module's code into another file

2 · Importing Modules

Python offers a few different ways to bring a module's code into your file:

Syntax	What it does
<code>import module</code>	Imports the whole module; access with <code>module.name</code>
<code>import module as alias</code>	Imports the module under a shorter alias
<code>from module import name</code>	Imports one specific function / class / variable
<code>from module import *</code>	Imports everything from the module (not recommended)

```
Python
1  import math
2  print(math.sqrt(16))      # 4.0
3
4  from math import sqrt
5  print(sqrt(16))          # 4.0
6
7  import math as m
8  print(m.pi)              # 3.14159...
```

■ **Tip:** Avoid `from module import *` in real projects — it makes it unclear where a name came from and can silently overwrite existing names.

3 · Packages

A **package** is a **directory** that groups multiple modules together. It must contain a special `__init__.py` file, which marks the directory as a package and can define dunder (double-underscore) attributes used to control its behaviour.

Part	Description
Package	A directory containing modules
<code>__init__.py</code>	Marks the directory as a package; can hold init / setup code
dunder methods	Special double-underscore names (e.g. <code>__init__</code> , <code>__name__</code>)

4 · Libraries

Libraries = Packages + Modules — a library is a collection of packages and modules bundled together for distribution and reuse (e.g. NumPy, Pandas).

■ *Modules live inside packages, and packages + modules together make up a library.*

5 · Namespaces & Scope (LEGB Rule)

A **namespace** is a mapping of names to objects. Python resolves a variable name using the **LEGB** order:

Scope	Description
Local	Names defined inside the current function
Enclosed	Names in an outer (enclosing) function — relevant to closures
Global	Names defined at the top level of a module / script
Built-in	Names pre-defined by Python itself (print, len, etc.)

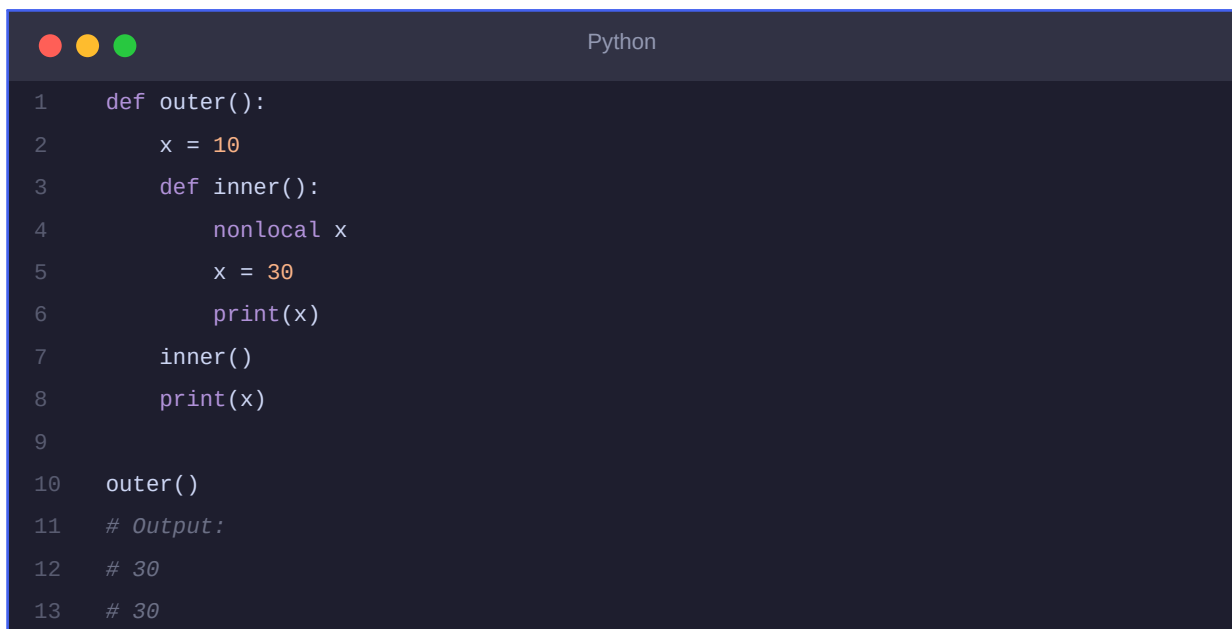
6 · global Keyword

The **global** keyword lets a function modify a variable that lives in the **global** (module-level) scope, instead of creating a new local variable.

```
Python
1  x = 10
2
3  def change_global():
4      global x
5      x = 50
6
7  change_global()
8  print(x)  # 50
```

7 · nonlocal Keyword

The **nonlocal** keyword lets a **nested** function modify a variable from its **enclosing (outer) function's** scope — it does not reach all the way to the global scope.

A screenshot of a Python IDE window titled "Python". The window has a dark blue background with light-colored text. The code is as follows:

```
1  def outer():
2      x = 10
3      def inner():
4          nonlocal x
5          x = 30
6          print(x)
7      inner()
8      print(x)
9
10 outer()
11 # Output:
12 # 30
13 # 30
```

■ **Tip:** Use **global** to change a global-scope variable from inside a function; use **nonlocal** to change an enclosing function's variable from a nested function.

8 · Errors & the Exception Hierarchy

An **error** is any **deviation from the correct / expected output** of a program — it interrupts normal execution unless it is handled.

Almost every error you will catch or define yourself inherits from **BaseException**, by way of the **Exception** class:

```
Hierarchy
1  BaseException
2      Exception
3          ValueError
4          TypeError
5          KeyError
6          IndexError
7          ZeroDivisionError
8          ... (your custom exceptions go here too)
```

9 · try / except / else / finally

Python's exception-handling block has four parts:

Block	When it runs
try	Code that might raise an exception
except	Runs only if an exception occurs inside try
else	Runs only if no exception occurred in try
finally	Always runs, whether or not an exception occurred

```
Python
1  try:
2      result = 10 / 2
3  except:
4      print('An exception occurred')
5  else:
6      print('No exception - result:', result)
7  finally:
8      print('This always runs')
```

10 · Catching Multiple Exceptions

A single **except** can catch several exception types at once using a tuple, or you can use separate **except** blocks to handle each one differently.

Tuple of exceptions

```
Python
1  try:
2      value = int(input('Enter a number: '))
3      result = 10 / value
4  except (ValueError, ZeroDivisionError) as e:
5      print('Invalid input or division by zero:', e)
```

Separate except blocks

```
Python
1  try:
2      value = int(input('Enter a number: '))
3  except ValueError:
4      print('Not a valid number')
5  except ZeroDivisionError:
6      print('Cannot divide by zero')
```

■ *except Exception as e captures the exception object itself, so you can inspect or print its message.*

11 · raise & assert

Use **raise** to manually trigger an exception when a condition you define is not met — typically inside a **try** block so it can be caught.

```
Python
1  if amount < 0:
2      raise ValueError('Amount cannot be negative')
```

assert is a quick sanity check: it raises **AssertionError** if the condition is False. It is mainly used for debugging and testing, not for handling expected runtime errors.

```
Python
1  assert amount >= 0, 'Amount cannot be negative'
```

12 · Custom Exceptions

You can create your **own exception type** by defining a class that inherits from **Exception**. This makes error handling more specific and descriptive.

```
Python
1  class InsufficientBalanceError(Exception):
2      pass  # to create our own exception
```

Using a custom exception

```
Python
1  def withdraw(balance, amount):
2      try:
3          if amount > balance:
4              raise InsufficientBalanceError('Not Enough Balance')
5          return balance - amount
6      except InsufficientBalanceError as e:
7          print('Error:', e)
```

■ Custom exceptions can also override `__init__` to store extra data, such as the amount that was missing.

13 · Common Built-in Exceptions

A quick reference for the exceptions you will run into most often:

Exception	Raised when...
<code>ValueError</code>	An argument has the right type but an invalid value
<code>TypeError</code>	An operation is applied to an object of the wrong type
<code>KeyError</code>	A dictionary key is not found
<code>IndexError</code>	A sequence index is out of range
<code>ZeroDivisionError</code>	A number is divided by zero
<code>FileNotFoundError</code>	A file or directory doesn't exist
<code>AttributeError</code>	An attribute reference or assignment fails
<code>ImportError</code> / <code>ModuleNotFoundError</code>	An import statement fails to find the module

Quick Reference — Modules & Exceptions

Tool	Purpose	Import needed?
<code>import module</code>	Bring a module's code into your script	No
<code>__init__.py</code>	Marks a directory as a package	No (auto-detected)
<code>global</code>	Modify a global-scope variable inside a function	No
<code>nonlocal</code>	Modify an enclosing function's variable in a nested function	No
<code>try / except</code>	Catch and handle exceptions	No
<code>else / finally</code>	Run code on success / always, around try-except	No
<code>raise</code>	Manually trigger an exception	No
<code>assert</code>	Quick debug check — raises <code>AssertionError</code> if False	No
<code>class X(Exception)</code>	Define your own custom exception type	No

```
Python
1  # Putting it together - custom exception with try / except / finally
2  class InsufficientBalanceError(Exception):
3      pass
4
5  def withdraw(balance, amount):
6      try:
7          if amount > balance:
8              raise InsufficientBalanceError('Not Enough Balance')
9              balance -= amount
10         except InsufficientBalanceError as e:
11             print('Transaction failed:', e)
12         else:
13             print('Withdrawal successful. New balance:', balance)
14         finally:
15             print('Transaction attempt finished.')
```

■ **Tip:** try / except / else / finally combined with a custom Exception subclass is a common pattern for clear, descriptive error handling in real-world Python code.